

State Tracking for Long-Horizon Interactive Narrative Generation with Large Language Models

Abstract

Large Language Models (LLMs) often struggle to maintain narrative coherence and rule adherence over extended multi-turn interactions, primarily due to inherent limitations in state tracking. This paper investigates whether the integration of an external "Observer LLM" can enhance the consistency of story generation compared to a standalone LLM architecture. We evaluate four distinct story simulation frameworks: (1) a standalone LLM functioning as the sole Game Master with implicit state tracking (hiding numerical values from output), (2) a standalone LLM with explicit state tracking (revealing numerical changes and consumed rules after each turn), (3) a deterministic programmatic state tracker, and (4) a collaborative framework utilizing an auxiliary Observer LLM to monitor and rectify state deviations. Experimental results demonstrate that relying solely on a single LLM for hidden state maintenance is unreliable, yielding average state tracking accuracy scores of only 0.10 (strict rule enforcement) and 0.11 (flexible rule adaptation). In contrast, the integration of an Observer LLM significantly improves performance, achieving average accuracy scores of 0.58 and 0.61, respectively. These findings suggest that an external observational mechanism is essential for mitigating state degradation in long-horizon narrative reasoning tasks.

Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities in long-context text generation and interactive dialogue, offering distinct advantages such as rapid, coherent drafting and versatile writing styles. However, despite their ability to produce fluent outputs, LLMs frequently struggle to maintain world-state coherence and causal consistency. During extended interactions, unresolved contradictions and accumulated logical errors can lead to narrative degradation, ultimately undermining the interpretability and meaningfulness of the generated story. Once the narrative ceases to align with the actual underlying state, the model drifts into hallucination. To address these limitations, prior research has explored incorporating external mechanisms to assist LLMs. For instance, Lewis et al. ([Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks](#)) introduced Retrieval-Augmented Generation (RAG) to inject external knowledge into knowledge-intensive tasks, while Gutiérrez et al. ([HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models](#)) proposed dynamic knowledge graphs to make complex states more accessible and comprehensible for LLMs. Recent advancements in Large Language Models have significantly improved interactive storytelling frameworks. For instance, Wu et al. demonstrated that integrating dramatic knowledge structures can substantially enhance narrative scaffolding and player immersion, while Pan et al. explored the use of knowledge graphs to preserve narrative consistency. Although these progressive environments provide powerful foundations for LLMs to generate compelling stories, a critical question remains underexplored: how foundational architectures perform specifically in terms of state-tracking fidelity and rule-adherence under controlled dynamics. As recent empirical findings by Ma et al. reveal, even in standard conversational settings, standalone LLMs frequently generate mutually contradictory plot developments due to an inherent lack of strict state maintenance. This vulnerability highlights the necessity of closely examining underlying tracking mechanisms before scaling up to more complex, long-horizon interactive narratives.

To address these limitations, this study utilizes [gpt-4o-mini](#) (OpenAI, 2024) as the Game Master LLM within a custom-built narrative simulation platform. The simulation scenario involves a spacecraft evacuation following an accident, where players must escape to an abandoned space mining station and transmit a distress signal before resource depletion. The world state comprises three core metrics: *power*, *oxygen*, and *stress*. Each player action consumes power and oxygen while accumulating stress; the simulation terminates once power or oxygen is exhausted or when stress exceeds its maximum threshold. While a deterministic baseline of resource consumption rules is predefined, the Game Master LLM is also permitted to dynamically introduce new rules outside the predefined rulebook. We design two experimental settings: a constrained setting where the Game Master cannot introduce new rules, and an open-ended setting where dynamic rule creation is permitted. Within these settings, we systematically evaluate four distinct frameworks: (1) a standalone LLM functioning as the sole Game Master with implicit state tracking (hiding numerical values from output), (2) a standalone LLM with explicit state tracking (revealing numerical changes and consumed rules after each turn), (3) a deterministic programmatic state tracker, and (4) a collaborative framework utilizing an auxiliary Observer LLM to monitor and rectify state deviations.

Related Work

2.1 Interactive Narratives and Game Master Challenges

In domains characterized by complete information and explicit rules, such as chess and Go, artificial intelligence has already matched or surpassed human proficiency. However, open-ended environments driven by natural language, epitomized by tabletop role-playing games like *Dungeons & Dragons* (D&D), present the next frontier for AI (Louis & Sutton, 2018). Evaluating AI systems within such highly open-ended settings—where dialogue must simultaneously reference background knowledge and adhere to complex rules—serves as a robust measure of linguistic and reasoning capabilities. Previous empirical analyses by Callison-Burch et al. (2022) revealed that when a Game Master LLM is forced to simultaneously manage narrative generation and rigorous state tracking, performance often degrades. This underscores the necessity of externalized support mechanisms to assist narrative models in preserving logical coherence and engaging plots.

2.2 Narrative Consistency and State Tracking Vulnerabilities

Maintaining strict adherence to established rules remains a persistent bottleneck for generative models. To quantify this, Ma et al. (2026) introduced the concept of *Narrative Commitment Preservation* to evaluate whether generated plots violate established constraints, demonstrating that even during standard conversational turns, standalone Game Master LLMs frequently produce mutually contradictory storylines. This tendency to drift stems from an inherent difficulty in maintaining persistent tracking of hidden world states over extended interactions.

2.3 External Memory and Knowledge Graph Integration

To mitigate these limitations, recent paradigms have increasingly incorporated external architectures to augment LLM decision-making. Park et al. (2023) introduced "Generative Agents" (the "Sandbox Village" framework), which simulate a memory stream consisting of observation, retrieval, and reflection mechanisms to provide external references for agent behavior. Similarly, Ammanabrolu et al. (2020) proposed integrating dynamic knowledge graphs into text-adventure games, enabling narrative LLMs to more effectively select rational actions, continuously update plot contexts, and enhance the overall interpretability of the generated outputs. Building upon these insights, our work explores how structured external mechanisms can effectively resolve state degradation in dynamic narrative environments.

2.4 Positioning of Our Work

Existing studies have demonstrated the benefits of external memory and structured knowledge for interactive narrative generation. However, our work focuses specifically on isolating and evaluating the state-tracking component under controlled resource dynamics and dynamically introduced rules.

Methodology

3.1 Simulation Environment and Narrative Setting

To evaluate narrative generation and state tracking, we implement a custom narrative simulation platform powered by `gpt-4o-mini` (OpenAI, 2024) acting as the central Game Master LLM. The simulation scenario places the player in a survival crisis: following a spacecraft accident, the player must evacuate to an abandoned space mining station and strategically manage their actions under severe resource scarcity.

The simulation environment is governed by three core state attributes: *power*, *oxygen*, and *stress*. The game terminates under two failure conditions: resource depletion (i.e., when either *power* or *oxygen* drops to zero, indicating a total failure of life support systems) or psychological collapse (i.e., when *stress* exceeds a threshold of 10 points). To survive, the player must issue conversational actions to explore the environment, perform repairs, and ultimately transmit a distress signal.

3.2 Initial States and Predefined Consumption Rules

Inspired by resource-constrained evaluation frameworks in strategic environments (Tracing LLM Reasoning Processes with Strategic Games: A Framework for Resource-Constrained Decision Making and Revision), we establish a controlled environment governed by three core states and a foundational set of consumption rules. Furthermore, we anticipate incorporating a Player LLM in future work to establish a fully automated game evaluation mechanism.

At the beginning of the simulation, the world state is initialized with 40 power, 20 oxygen, and 0 stress. Each player dialogue input represents one discrete simulation turn. State transitions are governed by a set of predefined deterministic consumption rules as follows:

- **Power:** Each turn incurs a baseline cost of $(- 2)$; specific actions induce additional costs, such as area illumination $(- 1)$, operating a plasma welder $(- 10)$, and transmitting a distress signal $(- 5)$.
- **Oxygen:** Each turn incurs a baseline cost of $(- 1)$; exploratory actions induce an additional cost of $(- 2)$.
- **Stress:** Each turn incurs a baseline accumulation of $(+ 1)$; actions performed in dark environments incur an additional $(+ 2)$.

Experiments and Evaluation

4.1 Experimental Settings

To comprehensively evaluate state-tracking efficacy, we conduct two distinct experimental settings, with each configuration executed across 5 independent trials:

1. **Constrained Setting:** The Game Master LLM is strictly prohibited from introducing consumption rules outside the predefined rulebook. The simulation is driven by a fixed sequence of 7 scripted player action turns.
2. **Open-Ended Setting:** The Game Master LLM is permitted to dynamically introduce new consumption rules not present in the predefined rulebook, provided that the impact of these rules is explicitly defined. These newly introduced rules are manually verified to determine whether they are properly factored into state tracking. This setting is driven by a fixed sequence of 8 scripted player action turns. According to manual trace calculations under static rules, the simulation should terminate at the 8th turn due to stress exceeding the maximum threshold. However, due to logging discrepancies, we unfortunately observed only a single instance of game termination independent of state values. This discrepancy prevented reliable evaluation of termination correctness and motivated us to focus the current analysis on per-turn state tracking rather than terminal-state prediction.

4.2 Evaluated Frameworks

We systematically evaluate four distinct architectural frameworks across the aforementioned experimental settings:

1. **Standalone LLM (Implicit):** The Game Master LLM is tasked with both narrative generation and implicit state tracking. Numerical changes are withheld until the game terminates. The required output format is structured as: *[Environment][New Rules]*.
2. **Standalone LLM (Explicit):** Similar to the first framework, the Game Master LLM handles narrative generation, but performs explicit state tracking. The current state and the specific rules applied are displayed at the end of every turn. The required output format is: *[Environment][Current State][Applied Rules][New Rules]*.
3. **Deterministic Programmatic State Tracker:** The Game Master LLM focuses solely on narrative generation, while state calculations are handled deterministically via Python code based on per-turn actions and fed back to the LLM to assist in subsequent story generation. The input/output formats are structured as follows: Player input comprises *[Observer's LastState Resolution][Player Action]*, while Game Master output consists of *[Environment][New Rules]*.
4. **Collaborative Framework with an Auxiliary Observer LLM:** Following the Game Master's narrative generation, the dialogue history between the player and the Game Master is passed to an auxiliary Observer LLM. The Observer computes the state updates and feeds them back to the Game Master to guide narrative progression. The formatting is structured as: Player input comprises *[Observer's LastState Resolution][Player Action]*, Game Master output consists of *[Environment][New Rules]*, and Observer output is formatted as *[State][Applied Rules][New Rules]*.

4.3 Results and Analysis

To quantify state-tracking performance, we evaluate the *Rule Integration Accuracy* across 5 independent trials for each framework. Accuracy is calculated by verifying whether each framework correctly computes state changes and applies newly introduced rules based on the player's actions per turn. The detailed results for both experimental settings are summarized below.

4.3.1 Constrained Setting Performance In the constrained setting (where no dynamic rules are introduced), the overall average accuracy across the three core attributes (Power, Oxygen, and Stress) for each framework is as follows:

1. **Standalone LLM (Implicit):** Power: 0.11, Oxygen: 0.00, Stress: 0.20 (Overall Average: **0.10**).
2. **Standalone LLM (Explicit):** Power: 0.26, Oxygen: 0.40, Stress: 0.23 (Overall Average: **0.30**).
3. **Deterministic Programmatic State Tracker:** Power: 0.46, Oxygen: 0.60, Stress: 0.57 (Overall Average: **0.54**).
4. **Collaborative Framework (Observer LLM):** Power: 0.51, Oxygen: 0.66, Stress: 0.57 (Overall Average: **0.58**).

Analysis: These results demonstrate that within this simulation platform, burdening a single LLM with both narrative generation and state maintenance fails to compute states accurately. Specifically, implicit state tracking yields near-zero reliability, particularly in maintaining oxygen metrics (0.00). While Framework (3) manages per-turn baseline consumption through deterministic code, it remains limited. In contrast, the auxiliary **Observer LLM (Framework 4)** successfully tracks state progression by reasoning over the dialogue history, achieving the highest baseline accuracy (0.58).

4.3.2 Open-Ended Setting Performance

In the open-ended setting (where dynamic rule creation is permitted), the overall average accuracies and rule integration metrics across the four frameworks are as follows:

1. Standalone LLM (Implicit): Power: 0.10, Oxygen: 0.05, Stress: 0.18 (Overall Average: 0.11). Dynamic Rules: 11 generated in total, with 6 effectively impacting state calculations.
2. Standalone LLM (Explicit): Power: 0.10, Oxygen: 0.30, Stress: 0.43 (Overall Average: 0.28). Dynamic Rules: 8 generated in total, with 1 effectively impacting state calculations.
3. Deterministic Programmatic State Tracker: Power: 0.23, Oxygen: 0.63, Stress: 0.65 (Overall Average: 0.50). Dynamic Rules: 15 generated in total, with 4 effectively impacting state calculations.
4. Collaborative Framework (Observer LLM): Power: 0.40, Oxygen: 0.70, Stress: 0.73 (Overall Average: 0.61). Dynamic Rules: 6 generated in total, with all 6 effectively impacting state calculations.

Analysis: Once the Game Master LLM is granted the freedom to dynamically introduce new rules, the Deterministic Programmatic State Tracker (Framework 3) struggles to incorporate these unprogrammed constraints, causing a sharp drop in Power tracking accuracy (plummeting from 0.46 down to 0.23). Interestingly, although Standalone LLM Explicit (Framework 2) outperforms its implicit counterpart in general state logging, it similarly fails to apply newly created rules to active state updates. In contrast, the Auxiliary Observer LLM (Framework 4) demonstrates superior adaptability and robustness; despite the introduction of dynamic rules, it maintains the highest overall accuracy (0.61) and successfully integrates 100% of the effective new rules (6 out of 6) into state updates. In our experimental setting, the Observer LLM achieved the highest state-tracking accuracy, particularly when dynamically introduced rules were present.

Conclusion & Future Work

5.1 Conclusion In this paper, we presented a dialogue-based simulation testbed designed to evaluate the state-tracking performance of four distinct frameworks across two experimental settings. Although each experiment was conducted over 5 independent trials, these preliminary results indicate that the simulation platform can reveal substantial differences in state-tracking behavior across architectural configurations. Specifically, our findings confirm that burdening a standalone LLM with simultaneous narrative generation and state maintenance is unreliable. Furthermore, the Observer LLM demonstrated greater adaptability to dynamically introduced rules than the deterministic tracker in our current setting.

Beyond quantitative metrics, our qualitative observations revealed intriguing failure modes in LLM narrative generation. For instance, in one trial set within the vacuum of an abandoned space mining station, the Game Master LLM generated a severe physical and logical contradiction by describing the player "opening a window, allowing outside oxygen to flood the station and secure a successful escape." This failure illustrates that state inconsistency is not limited to numerical bookkeeping; it can also manifest as violations of physical and causal constraints.

5.2 Future Work Building upon these insights, we intend to pursue several directions for future research:

1. **Diverse Narrative Backgrounds:** Expanding the simulation testbed to different thematic scenarios to test framework generalizability.
2. **Dynamic Knowledge Graphs:** Integrating structured knowledge graphs to enhance the system's external memory and long-term context retention.
3. **Automated Evaluation via Player LLMs:** Replacing scripted inputs with a Player LLM to establish a fully automated, end-to-end evaluation loop.
4. **Long-Horizon Dialogue Scaling:** Scaling the collaborative observer framework to handle extended multi-turn narrative sessions, thereby mitigating cumulative drift over long horizons.